Lecture-7

Under the Hood

From power button to a running program: booting, memory, and the JS engine



Execution Context



Booting a PC



Main Memory



JS Engine vs Node

TODAY

What we're covering



JS Execution Context

The Memory + Code split behind every running script



Booting a computer

Power → BSL → OS loaded into RAM → control handed over



Main Memory layout

How the OS slices RAM between the kernel and every app

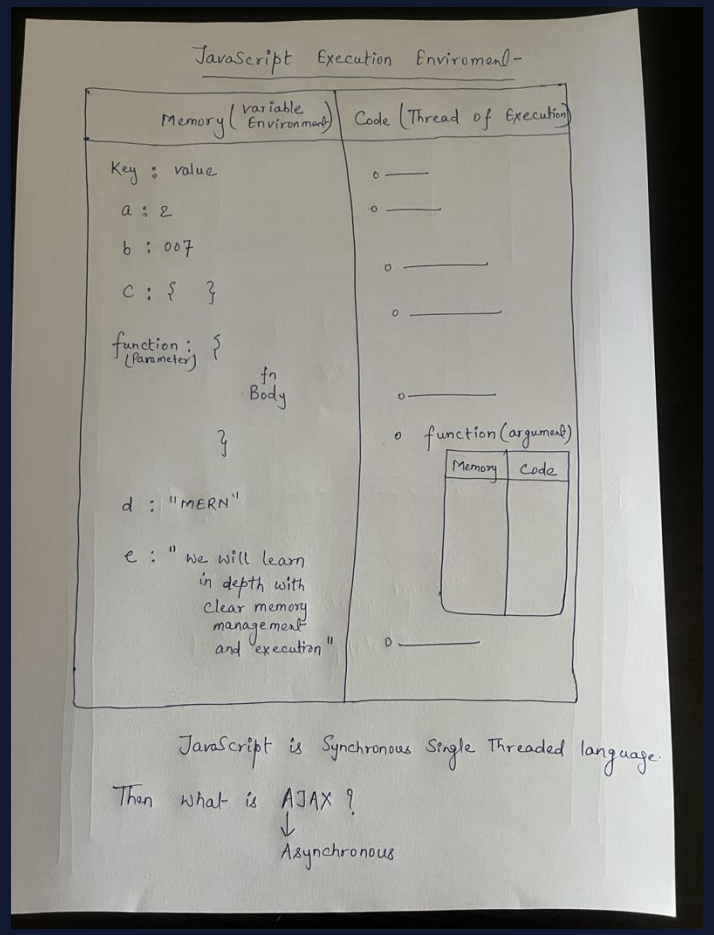


JS Engine vs Node.js

Why window exists in the browser but not in Node

EXECUTION CONTEXT

Every script runs inside Memory + Code



Whiteboard: Memory (Variable Environment) + Code (Thread of Execution)

Memory (Variable Environment)

Every variable and function is stored as a key : value pair BEFORE any code runs — a = 2, b = 007, c = {}, d = "MERN" ...

Code (Thread of Execution)

JS then executes line by line, top to bottom, using the values already sitting in memory.

This 2-part setup — Memory phase, then Code phase — is called the Execution Context, and JavaScript creates one every time it starts running any block of code.

EXECUTION CONTEXT

Calling a function creates a brand-new pair



`function(argument)`

The moment a function is called, JS doesn't just run its body — it opens a whole NEW Memory + Code pair, nested inside the one that called it.

The function gets its own private Memory for parameters/local variables, and its own Code thread to execute the function body — then that pair disappears once the function returns.

This nesting — a new Memory+Code table stacked on top of the previous one for every function call — is exactly what builds the JS Call Stack.

A QUICK PUZZLE

JS is synchronous & single-threaded... so what is AJAX?

JavaScript is a Synchronous, Single-Threaded language.

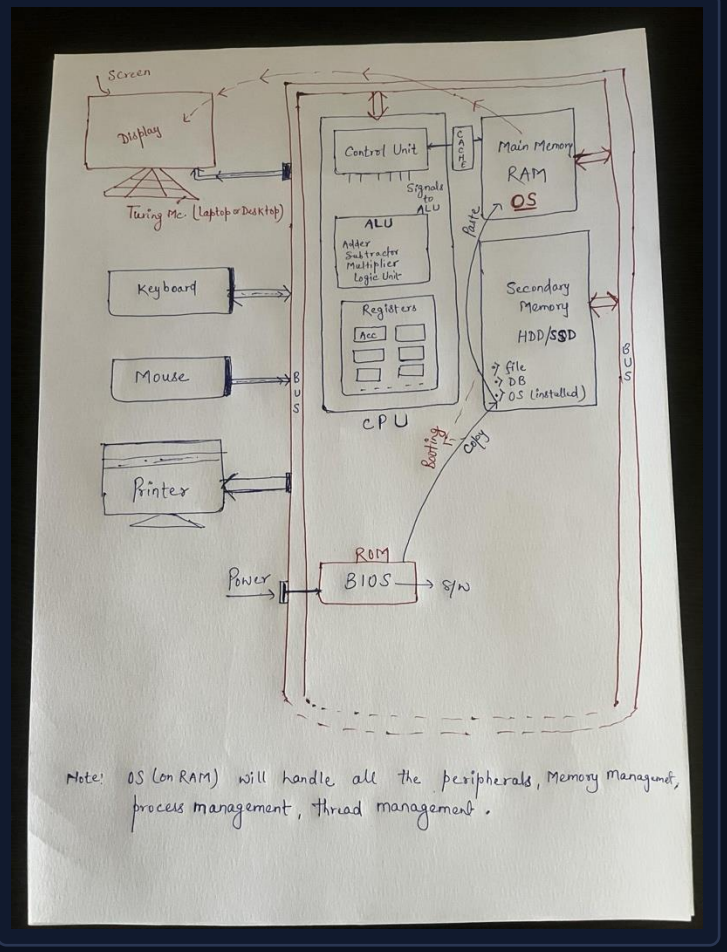
↓

Then what is AJAX? → Asynchronous.

The engine itself never runs two lines at once — but the environment around it (browser or Node) can hand off slow work elsewhere and come back later. That's the puzzle we'll unpack over the next few lectures.

ARCHITECTURE

Inside the machine: CPU, memory & peripherals



Whiteboard: CPU, RAM, Secondary storage, BSL, and peripherals

CPU = Control Unit (directs everything) + ALU (does the actual math/logic) + Registers (tiny, ultra-fast storage)

Main Memory (RAM) = fast but temporary — holds the OS and running programs

Secondary Memory (HDD/SSD) = slow but permanent — files, databases, and the installed OS

Correction: the ROM chip labelled "BIOS" in the photo should read BSL — the BootStrap Loader — the tiny fixed program that actually kicks off booting.

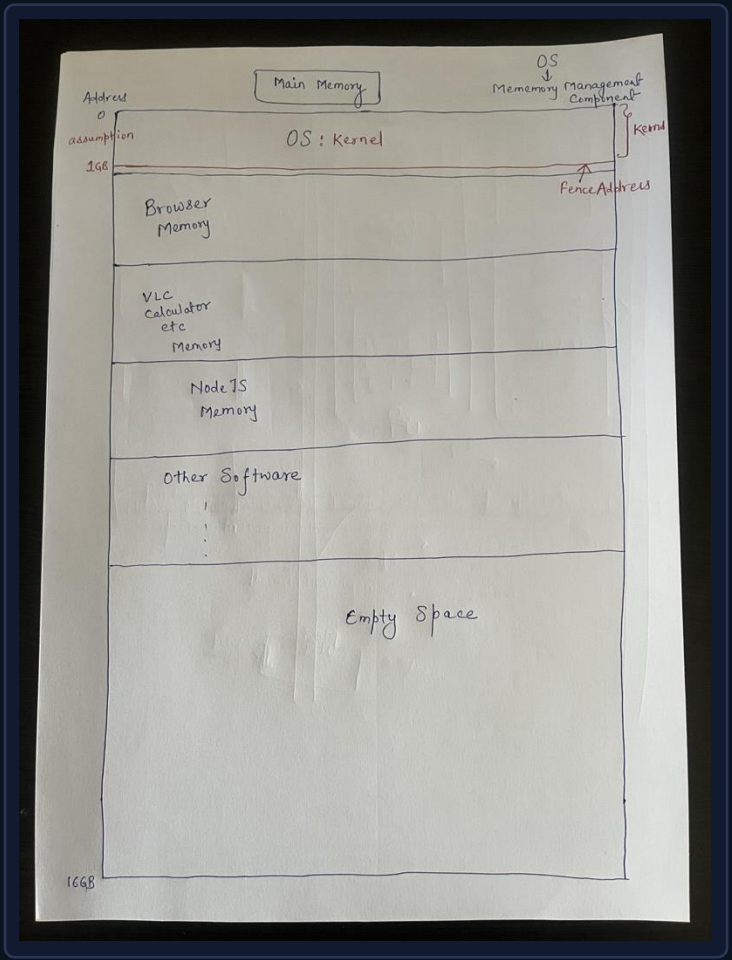
BOOTING

From power button to a running OS

1	**Power on**	Electricity reaches the motherboard and the CPU
2	**ROM runs the BSL**	The BootStrap Loader — a small fixed program permanently stored in ROM — starts executing
3	**BSL copies the OS**	The kernel is copied from Secondary Memory (HDD/SSD) into Main Memory (RAM)
4	**OS takes control**	The OS now manages peripherals, memory, processes, and threads

MEMORY LAYOUT

How the OS slices up RAM



Whiteboard: OS Kernel, Fence Address, then Browser / Node.js / other app memory

OS : Kernel sits at the very start of RAM (address 0), reserved and protected by the OS's Memory Management Component.

Fence Address marks the boundary below which user programs may never write — it protects the kernel from being overwritten.

Below the fence: every running app — Browser, Node.js, VLC, Calculator — gets its own separate slice, with empty space left over for whatever opens next.

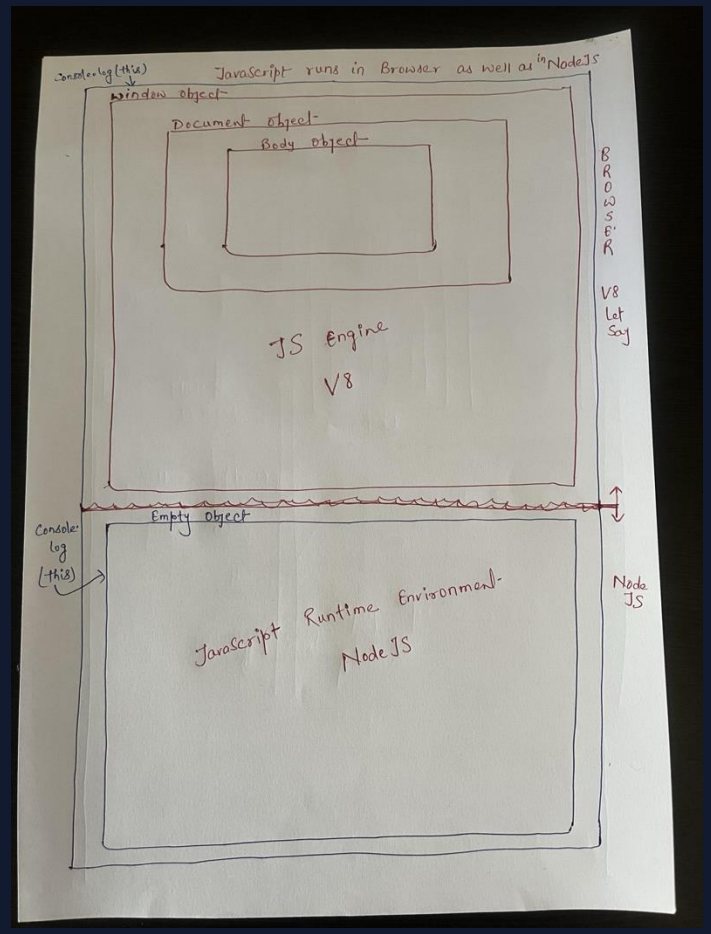
MEMORY LAYOUT

Every app is isolated — and so is every computation

Memory allocation	Handled by the OS (its Memory Management Unit) — this is why total memory usage is a space-complexity question, not a CPU one
Computation	Handled by the CPU — every ALU operation your program demands (add, compare, multiply…) counts as one unit of computation
Isolation	Browser memory, Node.js memory, VLC's memory, Calculator's memory — all live in separate slices; one app crashing shouldn't corrupt another's

JS EVERYWHERE

The same JS engine, two very different homes



Whiteboard: window object (Browser) vs the empty object (Node.js)

In the Browser

The V8 engine sits INSIDE a window object, which contains document → body, and all the DOM APIs. console.log(this) at the top level prints window.

In Node.js

The SAME V8 engine runs, but there's no window — Node wraps your file in an empty object instead. console.log(this) here prints {}.

JS EVERYWHERE

JS Engine vs Node.js — what's the actual difference?

	Browser (JS Engine + Browser APIs)	Node.js (JS Engine + Node APIs)
Engine	`V8 (or SpiderMonkey, etc.)`	`V8 — same engine!`
Global object	`window`	`an empty {} (module wrapper)`
DOM access	`document, body, getElementById...`	`Not available — no window/document`
File/OS access	`Not available (sandboxed)`	`fs, http, path, process...`

RECAP

What we covered today



Execution Context

Memory phase, then Code phase — for every script and every function call



Booting

Power → ROM's BSL → OS copied into RAM → OS takes control



Main Memory layout

Kernel protected by a fence address; every app gets its own slice



JS Engine vs Node.js

Same V8 engine — window in the browser, an empty object in Node

Thank You !